JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class Observable

java.lang.Object
    java.util.Observable

```
public class Observable
extends Object
```

This class represents an observable object, or "data" in the model-view paradigm. It can be subclassed to represent an object that the application wants to have observed.

An observable object can have one or more observers. An observer may be any object that implements interface `Observer`. After an observable instance changes, an application calling the `Observable`'s `notifyObservers` method causes all of its observers to be notified of the change by a call to their `update` method.

The order in which notifications will be delivered is unspecified. The default implementation provided in the Observable class will notify Observers in the order in which they registered interest, but subclasses may change this order, use no guaranteed order, deliver notifications on separate threads, or may guarantee that their subclass follows this order, as they choose.

Note that this notification mechanism has nothing to do with threads and is completely separate from the `wait` and `notify` mechanism of class `Object`.

When an observable object is newly created, its set of observers is empty. Two observers are considered the same if and only if the `equals` method returns true for them.

Since:

`JDK1.0`

See Also:

notifyObservers(), notifyObservers(java.lang.Object), Observer, Observer.update(java.util.Observable, java.lang.Object)

Constructor Summary

Constructors

Constructor	Description
Observable()	Construct an Observable with zero Observers.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
void	addObserver(Observer o)	Adds an observer to the set of observers for this object, provided that it is not the same as some observer already in the set.
protected void	clearChanged()	Indicates that this object has no longer changed, or that it has already notified all of its observers of its most recent change, so that the hasChanged method will now return false.
int	countObservers()	Returns the number of observers of this Observable object.
void	deleteObserver(Observer o)	Deletes an observer from the set of observers of this object.
void	deleteObservers()	Clears the observer list so that this object

		no longer has any observers.
boolean	**hasChanged**()	Tests if this object has changed.
void	**notifyObservers**()	If this object has changed, as indicated by the `hasChanged` method, then notify all of its observers and then call the `clearChanged` method to indicate that this object has no longer changed.
void	**notifyObservers**(**Object** arg)	If this object has changed, as indicated by the `hasChanged` method, then notify all of its observers and then call the `clearChanged` method to indicate that this object has no longer changed.
protected void	**setChanged**()	Marks this `Observable` object as having been changed; the `hasChanged` method will now return `true`.

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Detail

Observable

```
public Observable()
```

Construct an Observable with zero Observers.

Method Detail

addObserver

```
public void addObserver(Observer o)
```

Adds an observer to the set of observers for this object, provided that it is not the same as some observer already in the set. The order in which notifications will be delivered to multiple observers is not specified. See the class comment.

Parameters:

`o - an observer to be added.`

Throws:

`NullPointerException - if the parameter o is null.`

deleteObserver

```
public void deleteObserver(Observer o)
```

Deletes an observer from the set of observers of this object. Passing `null` to this method will have no effect.

Parameters:

`o - the observer to be deleted.`

notifyObservers

```
public void notifyObservers()
```

If this object has changed, as indicated by the `hasChanged` method, then notify all of its observers and then call the `clearChanged` method to indicate that this object has no longer changed.

Each observer has its `update` method called with two arguments: this observable object and `null`. In other words, this method is equivalent to:

```
notifyObservers(null)
```

See Also:

`clearChanged()`, `hasChanged()`, `Observer.update(java.util.Observable, java.lang.Object)`

notifyObservers

```
public void notifyObservers(Object arg)
```

If this object has changed, as indicated by the `hasChanged` method, then notify all of its observers and then call the `clearChanged` method to indicate that this object has no longer changed.

Each observer has its `update` method called with two arguments: this observable object and the `arg` argument.

Parameters:

`arg - any object.`

See Also:

`clearChanged()`, `hasChanged()`, `Observer.update(java.util.Observable, java.lang.Object)`

deleteObservers

```
public void deleteObservers()
```

Clears the observer list so that this object no longer has any observers.

setChanged

```
protected void setChanged()
```

Marks this `Observable` object as having been changed; the `hasChanged` method will now return `true`.

clearChanged

```
protected void clearChanged()
```

Indicates that this object has no longer changed, or that it has already notified all of its observers of its most recent change, so that the `hasChanged` method will now return `false`. This method is called automatically by the `notifyObservers` methods.

See Also:

`notifyObservers()`, `notifyObservers(java.lang.Object)`

hasChanged

```
public boolean hasChanged()
```

Tests if this object has changed.

Returns:

`true if and only if the setChanged method has been called more recently than the clearChanged method on this object; false otherwise.`

See Also:

`clearChanged()`, `setChanged()`

countObservers

```
public int countObservers()
```

Returns the number of observers of this `Observable` object.

Returns:

`the number of observers of this object.`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.